Cascade, specificity, and inheritance

You have now written enough CSS for rules to overlap, which means one element might match several selectors.

Here is a small example. You may have a button that looks like this:

```html
<button class="button primary-button">click me</button>
```

This button can be matched by the `button` element selector, the `.button` class selector, and the `.primary-button` class selector.

```css
button {
  background-color: gray;
}

.button {
  background-color: #e5e7eb;
}

.primary-button {
  background-color: #2563eb;
}
```

All three rules set `background-color` for the button. How does CSS decide which `background-color` wins?

CSS is not guessing. The browser follows a decision process called the cascade.

Start with overlapping rules

Use a small two-file page for this cascade exercise: `index.html` and `style.css`.

Start with HTML that lets several rules overlap:

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Cascade practice</title>
    <link rel="stylesheet" href="style.css">
```

```
  </head>
  <body>
    <main class="page">
      <h1>Cascade practice</h1>
      <p class="lead" id="intro">This paragraph matches several rules.</p>
      <p>This paragraph helps us compare inherited and direct styles.</p>
      <button class="button primary-button">Save changes</button>
    </main>
  </body>
</html>
```

Add the CSS with overlapping rules:

css

```
body {
  font-family: system-ui, sans-serif;
  background-color: #f9fafb;
}

.page {
  max-width: 720px;
  margin: 40px auto;
  color: #111827;
}

p {
  color: #444;
}

.lead {
  color: #2563eb;
  font-size: 20px;
}

#intro {
  color: #b91c1c;
}

.button {
  padding: 12px 16px;
  border: 0;
  border-radius: 6px;
}

.primary-button {
  background-color: #2563eb;
  color: white;
}
```

Previous Lesson Next Lesson

Open the page. The first paragraph matches `p`, `.lead`, and `#intro`, and all three rules set `color`. The browser has to choose one.

The cascade chooses a value

The cascade only matters when two or more declarations try to set the same property on the same element.

This is not a conflict:

css

```
.lead {
  color: #2563eb;
}
```

```css
.lead {
  font-size: 20px;
}
```

One declaration sets `color`, and the other sets `font-size`, so the paragraph can use both.

This is a conflict:

```css
.lead {
  color: #2563eb;
}

#intro {
  color: #b91c1c;
}
```

Both declarations set `color` on the same element. The browser needs to decide which value wins.

The cascade looks at three things:

1. **Specificity:** how strong each selector is. A more specific selector wins over a less specific one. In the example above, `#intro` is stronger than `.lead`, which is stronger than `p`, so the lead paragraph is red.
2. **Source order:** when two rules have the same specificity, the one that appears later in the CSS file wins. The button in the practice page has two class selectors, `.button` and `.primary-button`, so source order decides which shared background color shows up.
3. **Importance:** a declaration marked `!important` can beat normal declarations. You can force CSS to use the styles from a less specific selector using the `!important` declaration. For example you can have `color: #2563eb !important;` to make CSS force the color. It is meant for rare overrides, not everyday styling.

The next few sections walk through each one in turn, starting with specificity.

Specificity is selector strength

Specificity means **selector strength**. A more specific selector wins over a less specific selector when both set the same property on the same element.

Specificity exists so you can write broad rules and then override them for specific cases without deleting the broad rule.

For example, `p { color: #444; }` can style normal paragraphs, while `.lead { color: #2563eb; }` can make one kind of paragraph stand out. The

cascade makes that layering predictable.

In the practice page, the lead paragraph matches all of these:

```css
p {
  color: #444;
}

.lead {
  color: #2563eb;
}

#intro {
  color: #b91c1c;
}
```

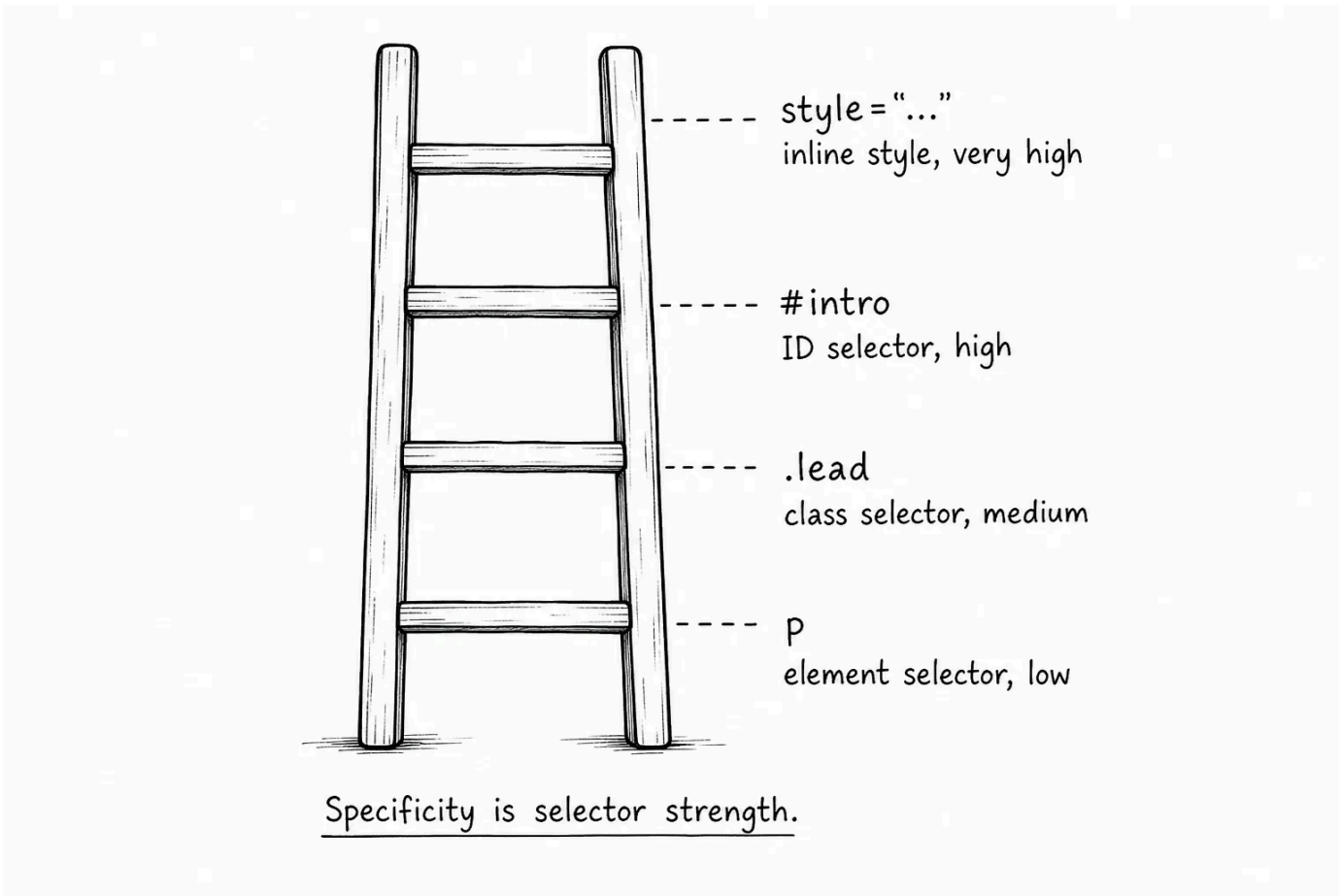
The paragraph is red because `#intro` is the most specific selector in that group.

A useful beginner ranking looks like this:

Selector type	Example	Rough strength
Element selector	`p`	Low
Class selector	`.lead`	Medium
ID selector	`#intro`	High
Inline style	`style="color: red;"`	Very high

That table is not the full specificity system, but it is enough for early debugging.

We are using `#intro` here only to make specificity easy to see. In real projects, you would usually add another class instead of styling with an ID.



Source order breaks ties

Source order matters when selectors have the same specificity.

Look at the following button rules:

```css
.button {
  background-color: #e5e7eb;
  color: #111827;
}

.primary-button {
  background-color: #2563eb;
  color: white;
}
```

The button has both classes in its HTML:

```html
<button class="button primary-button">Save changes</button>
```

Both `.button` and `.primary-button` are class selectors, so they have the same specificity. For shared properties like `background-color` and `color`, the later rule wins.

That is why the button is blue with white text.

If you move `.primary-button` above `.button`, the shared `.button` rule comes later and wins those same properties. The button becomes gray again.

Source order does not help if specificity is different. If you move `.lead` below `#intro`, the lead paragraph is still red because `#intro` is more specific.

> **Order matters when strength is the same - Order matters when strength is the same** - Source order is not random. If two matching declarations are equally specific, the one that appears later in the CSS wins.

One thing to note about class selectors: as discussed, if you have two selectors with the same specificity that target the same element (e.g. `.button` and `.primary-button`), CSS applies the rule that appears later in the stylesheet.

If you don't want to rely on the order of the rules, you can make a selector more specific. For example, `button.primary-button` or `.parent .primary-button` are more specific than `.primary-button` alone, so their styles would take precedence even if competing rules appear later in the stylesheet.

More specific is not always better

Once you learn specificity, it is tempting to make selectors stronger whenever something does not work.

This selector is strong, but it is tied to the current HTML structure:

```css
main .page p.lead {
  color: #2563eb;
}
```

If the paragraph moves, or the class is reused somewhere else, the selector may stop being helpful.

This is usually easier to understand and reuse:

```css
.lead {
  color: #2563eb;
}
```

Use enough specificity to target the thing clearly. Avoid adding extra element names, IDs, or ancestor chains just to win.

> **Do not win by making selectors huge** - If you keep making selectors longer just to override earlier CSS, the stylesheet becomes harder to change. First ask whether the class name, source order, or component structure can be simpler.

Inheritance passes some styles down

One important thing that trips beginners up is inheritance. Some CSS properties applied to a parent element can pass down to child elements automatically.

In the practice page, `.page` sets a text color:

```css
.page {
  color: #111827;
}
```

That color becomes the default for text inside `.page` unless a more direct rule sets another value.

In the starting CSS, paragraphs also have a direct rule:

```css
p {
  color: #444;
}
```

That means the paragraphs use the direct `p` color, not the inherited `.page` color. If you remove the `p` color, the paragraphs fall back to the inherited `.page` color.

Text-related properties often inherit, which is why this kind of rule is so useful:

```css
body {
  font-family: system-ui, sans-serif;
}
```

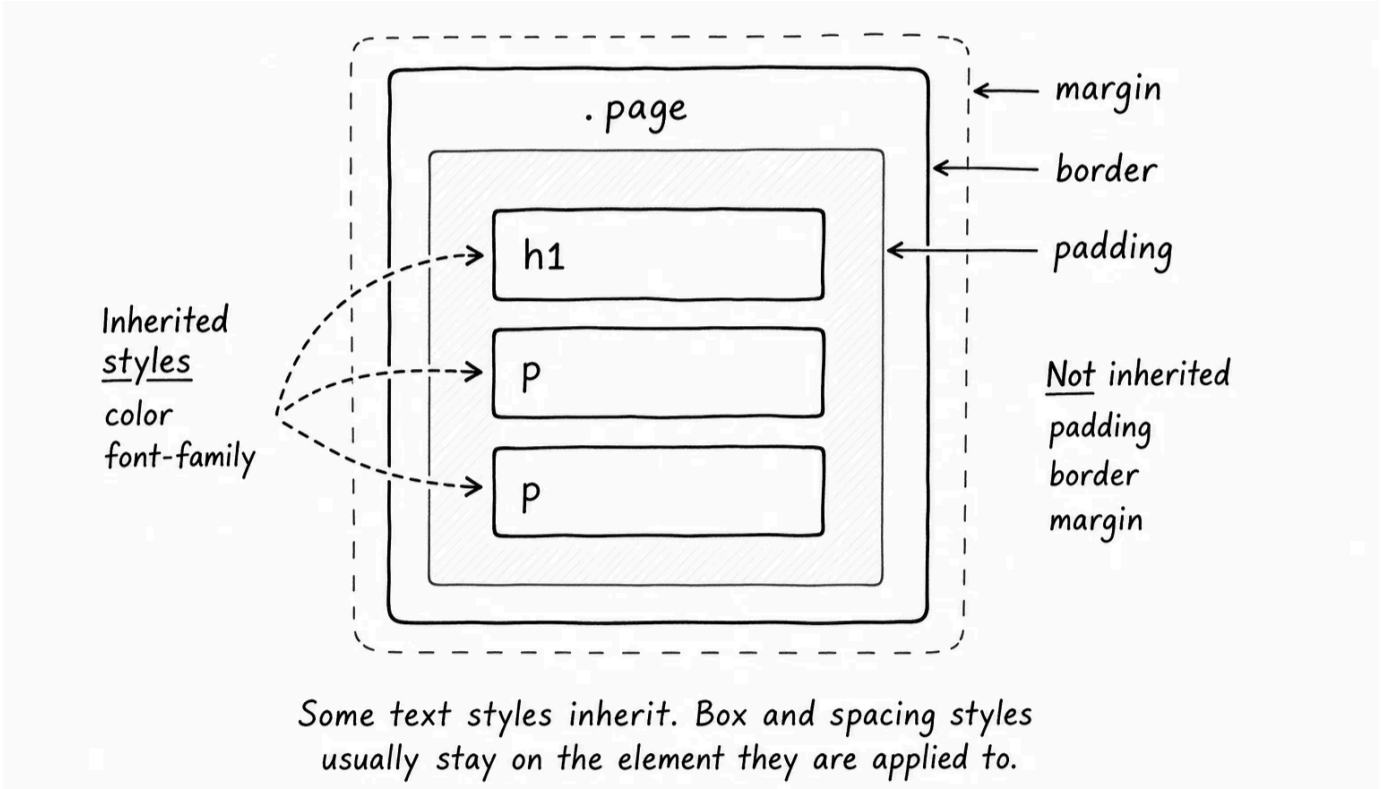
That is why you can set `font-family` once on `body` and most text on the page uses it.

Links are a common inheritance surprise. A link may stay browser-blue even when nearby text inherits a different color, because the browser gives links their own direct color by default. Inheritance is a fallback; direct declarations on the element win.

Box and layout properties usually do not inherit:

```css
.page {
  padding: 24px;
  border: 1px solid #e5e7eb;
}
```

The children inside `.page` do not each get their own padding and border. Those styles belong to the `.page` box.



Some text styles inherit. Box and spacing styles usually stay on the element they are applied to.

Direct styles beat inherited styles

Inheritance gives children a default value, but a direct declaration on the child wins.

Look at this CSS:

```css
.page {
  color: #111827;
}

p {
  color: #444;
}
```

The `p` rule wins for paragraphs because it directly targets the paragraph. The inherited `.page` color is just the fallback.

The same idea applies to `.lead` and `#intro`: they directly target the first paragraph, so they beat inherited text color from `.page`.

Inherited values are defaults for descendants, not locked-in commands.

Crossed-out styles are clues

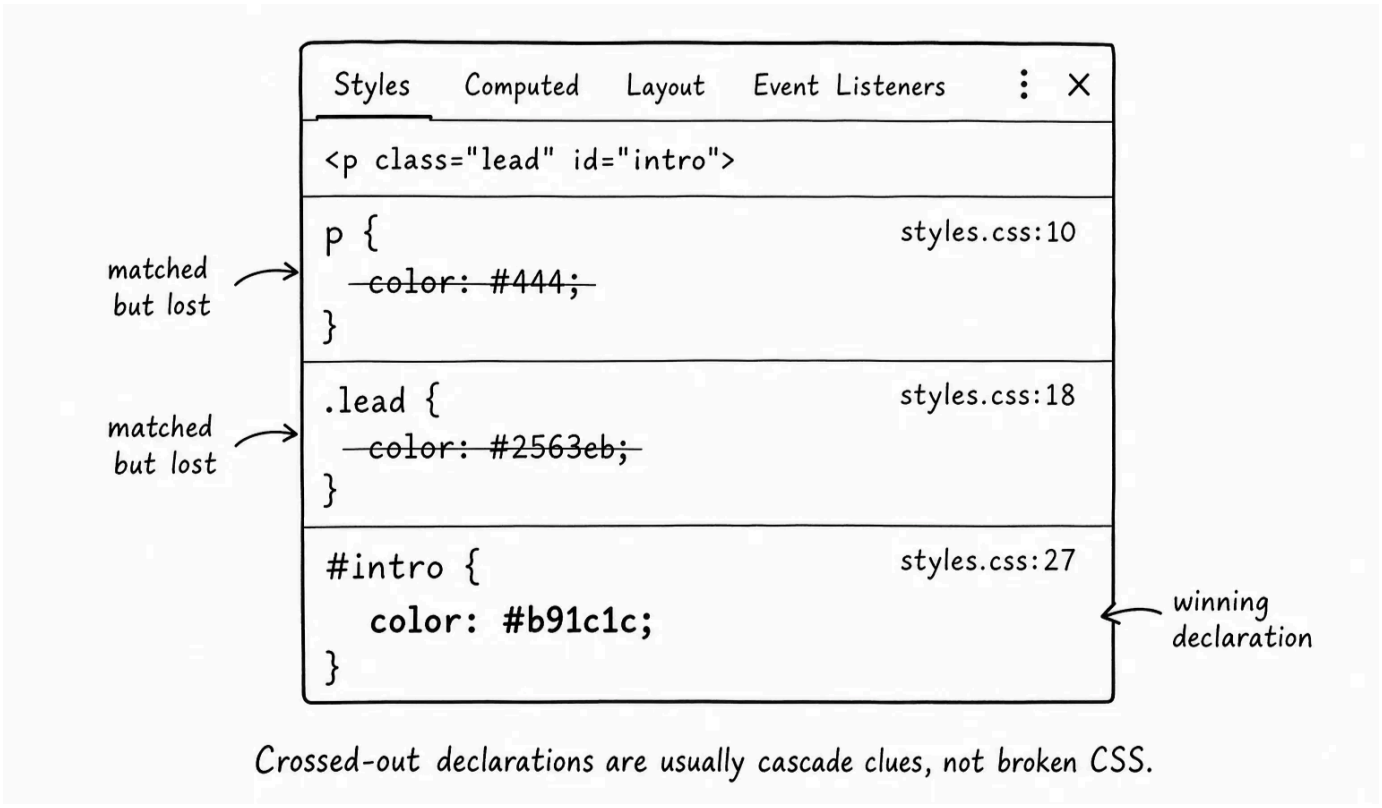
When a declaration loses, DevTools often shows it crossed out.

Inspect the lead paragraph in the browser. You may see the `p` color and `.lead` color crossed out, while the `#intro` color is active.

That does not mean the `p` selector failed. It matched, but its `color` declaration lost to a more specific rule.

This distinction matters when you debug:

- If a rule does not appear in DevTools, the selector probably did not match.
- If a declaration appears crossed out, the selector matched, but another declaration won.



Crossed-out declarations are usually cascade clues, not broken CSS.

Be careful with important

You can force a declaration to win with `!important`:

```css
.lead {
  color: #047857 !important;
}
```

That tells the browser to treat this declaration as more important than normal declarations.

`!important` can be useful when you are dealing with third-party styles or a very specific override. But it is a bad everyday habit. Once you use it, normal source order and specificity become harder to reason about, and future CSS may need even more `!important` to override it.

> ⚠ **Do not use important as a normal fix** - If a style is not applying, inspect the element first. Check whether your selector matches, whether another rule is more specific, and whether source order is the issue. Reach for `!important` only when you understand what you are overriding.

Debug the winning declaration

When a CSS value is not what you expected, debug it in this order:

1. Inspect the element in DevTools.
2. Find the property you care about, such as `color` or `background-color`.
3. Look for all matching rules that set that property.
4. Check whether any declaration is crossed out.
5. Notice whether any declaration uses `!important`.
6. Compare specificity: element, class, ID, inline style.
7. If specificity is the same, check which declaration appears later.
8. Check whether the value is inherited from a parent.

This sounds like a lot, but DevTools puts most of it in one place. The key habit is to inspect the actual element instead of guessing from the stylesheet.

Try it

Now make predictions before each change and use DevTools to check the cascade:

- Inspect the lead paragraph in DevTools. Find which `color` declaration wins.
- Move the `p` rule below the `.lead` rule. Predict what color the lead paragraph will be, then reload. It should stay red because `#intro` is more specific.
- Remove `id="intro"` from the lead paragraph. Predict whether `p` or `.lead` will win, then reload. `.lead` should win because a class is more specific than an element selector.
- Move `.primary-button` above `.button`. Predict the button background, then reload. The button should lose its blue background because the two class selectors have equal specificity and `.button` now comes later.
- Add `color: #047857;` to `.page`, then remove the direct `color` rules from `p`, `.lead`, and `#intro`. Predict where the paragraphs get their color, then reload. They should inherit the color from `.page`.
- Add `!important` to one `color` declaration. Predict whether it will win, inspect the element, then remove it again. Notice how quickly it makes the normal rules harder to read.

The goal is to see that CSS is following rules. When a value surprises you, DevTools can show which declaration won and which declarations lost.

Knew **0 Items** Learnt **0 Items** Skipped **0 Items** ResetProgress

Test Your Understanding

What is the cascade?

Click to Reveal the Answer

Already Know that | Didn't Know that | Skip Question

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →